

PICT

Pairwise Independent Combinatorial Testing tool

jacekcz@microsoft.com

Traduction en français : Pascal DELAMAIRE-02/2025

PICT génère des cas de test et des configurations de test. Avec PICT, vous pouvez générer des tests plus efficaces que ceux créés manuellement et en une fraction du temps requis par une conception manuelle des cas de test.

PICT fonctionne en ligne de commande. Préparez un fichier modèle décrivant les paramètres de l'interface (ou un ensemble de configurations ou de données) que vous souhaitez tester, et PICT génère un ensemble compact de choix de valeurs de paramètres qui représentent les cas de test nécessaires pour obtenir une couverture combinatoire complète.

Par exemple, pour créer une suite de tests pour la création de partitions de disque, le domaine peut être décrit par les paramètres suivants : Partition Type, Partition Size, File System, Format Method, Cluster Size, et Compression. (Type de partition, Taille de la partition, Système de fichiers, Méthode de formatage, Taille des clusters et Compression). Chaque paramètre comporte un nombre fini de valeurs possibles. Par exemple, la compression peut uniquement être activée (On) ou désactivée (Off), tandis que les autres paramètres sont caractérisés par une liste finie de valeurs.

```
Type:          Single, Span, Stripe, Mirror, RAID-5
Size:          10, 100, 500, 1000, 5000, 10000, 40000
Format method: Quick, Slow
File system:   FAT, FAT32, NTFS
Cluster size:  512, 1024, 2048, 4096, 8192, 16384, 32768, 65536
Compression:  On, Off
```

Pour un tel modèle, des milliers de cas de test possibles peuvent être générés. Il serait difficile de tous les tester dans un délai raisonnable. Plutôt que d'essayer de couvrir toutes les combinaisons possibles, nous nous concentrons sur le test de toutes les paires de valeurs possibles. Par exemple, `{Type:Single, File system:FAT}` est une paire de valeurs, tout comme `{Size:10, Format method:Slow}`. Par conséquent, un seul cas de test peut couvrir plusieurs paires.

Des recherches montrent que tester toutes les paires est une alternative efficace et bien moins coûteuse aux tests exhaustifs.

Usage

PICT est un outil en ligne de commande qui accepte un fichier modèle au format texte brut en entrée et génère un ensemble de cas de test.

Usage: `pict model [options]`

```
Options:
/o:N|max - Order of combinations (default: 2)
/d:C     - Separator for values (default: ,)
/a:C     - Separator for aliases (default: |)
/n:C     - Negative value prefix (default: ~)
/e:file  - File with seeding rows
```

```
/r[:N] - Randomize generation, N - seed
/c      - Case-sensitive model evaluation
/s      - Show model statistics
```

Fichier modèle

Un modèle est un fichier texte brut composé des sections suivantes :

```
parameter definitions
[sub-model definitions]
[constraint definitions]
```

Les sections doivent toujours être spécifiées dans l'ordre indiqué ci-dessus. Tous les paramètres doivent être définis en premier, suivis des sous-modèles, puis des contraintes.

La définition des paramètres est obligatoire, tandis que les autres parties sont facultatives.

Les sections ne nécessitent aucun séparateur particulier entre elles.

Des lignes vides peuvent être insérées n'importe où. Les commentaires sont autorisés et doivent être précédés du caractère #.

Pour créer un fichier modèle de base, il suffit de lister les paramètres — chacun sur une ligne distincte — avec les valeurs correspondantes séparées par des virgules :

```
<ParamName> :<Value1>, <Value2>, <Value3>, ...
```

Exemple:

```
#
# This is a sample model for testing volume creation
#

Type:          Primary, Logical, Single, Span, Stripe, Mirror, RAID-5
Size:          10, 100, 500, 1000, 5000, 10000, 40000
Format method: quick, slow
File system:    FAT, FAT32, NTFS
Cluster size:  512, 1024, 2048, 4096, 8192, 16384, 32768, 65536
Compression:   on, off
```

A comma is the default separator but you can specify a different one using /d option.

Par défaut, PICT génère une suite de tests pairwise (toutes les paires de valeurs sont couvertes).

Toutefois, il est possible de modifier cet ordre en utilisant l'option /o avec une valeur supérieure à deux.

Par exemple, lorsque l'option /o:3 est spécifiée, la suite de tests couvrira toutes les triplets de valeurs. Cela entraînera la génération d'un plus grand nombre de tests, mais rendra potentiellement la suite de tests plus efficace.

L'ordre maximal pour un modèle est égal au nombre total de paramètres, ce qui aboutira à une suite de tests exhaustive. En suivant le même principe, spécifier /o:1 produira une suite de tests qui couvre uniquement toutes les valeurs individuelles (combinaisons de 1).

Format de sortie

Les cas de test sont imprimés sur le flux de sortie standard. La première ligne de la sortie contient les noms des paramètres. Chaque ligne suivante représente un cas de test généré. Les valeurs de

chaque ligne sont séparées par une tabulation. Ainsi, en redirigeant la sortie vers un fichier, on obtient un format de valeurs séparées par des tabulations (TSV).

Si le modèle et les options fournies à l'outil ne changent pas, chaque exécution produira exactement le même résultat. Toutefois, il est possible de randomiser la génération des tests en utilisant l'option /r.

Une génération aléatoire affichera la graine utilisée pour cette exécution particulière dans le flux de sortie des erreurs.

Cette graine peut ensuite être réutilisée avec l'option /r:seed pour rejouer exactement la même génération de tests.

Différentes valeurs de graine aléatoire peuvent produire un nombre total de cas de test légèrement différent. Cela s'explique par la complexité du problème d'optimisation des combinaisons n-way, pour lequel PICT et d'autres outils utilisent des heuristiques. Ces heuristiques sont déterministes, mais dépendent des conditions initiales. Parfois, l'algorithme parvient à regrouper toutes les combinaisons souhaitées en un nombre plus réduit de cas de test. Des variations de l'ordre de 5 % à 10 % sont courantes.

Toutes les erreurs, les messages d'avertissement et les informations auxiliaires sont imprimés sur le flux de sortie des erreurs.

Contraintes

Les contraintes expriment les limitations inhérentes au domaine modélisé. Dans l'exemple ci-dessus, l'une des paires qui apparaîtra dans au moins un cas de test est {File system:FAT, Size:5000}. {Système de fichiers : FAT, Taille : 5000}. Or, en pratique, le système de fichiers FAT ne peut pas être appliqué sur des volumes supérieurs à 4 096 Mo.

Il est important de noter que l'on ne peut pas simplement supprimer ces cas de test invalides des résultats, car un cas de test contenant une combinaison non valide peut également inclure d'autres paires valides qui ne seraient autrement pas testées.

Plutôt que de perdre ces paires valides, il est préférable d'éliminer les combinaisons interdites dès la génération des tests.

Dans PICT, cela peut être réalisé en spécifiant des contraintes, par exemple :

```
Type:           Primary, Logical, Single, Span, Stripe, Mirror, RAID-5

Size:           10, 100, 500, 1000, 5000, 10000, 40000
Format method:  quick, slow
File system:    FAT, FAT32, NTFS
Cluster size:   512, 1024, 2048, 4096, 8192, 16384, 32768, 65536
Compression:   on, off
```

```
IF [File system] = "FAT"      THEN [Size] <= 4096;
IF [File system] = "FAT32"   THEN [Size] <= 32000;
```

Contraintes Conditionnelles

Un terme `[parameter] relation value` constitue une unité atomique d'une expression de contrainte. Les relations suivantes peuvent être utilisées : `=`, `<>`, `>`, `>=`, `<`, `<=`, et `LIKE`.

`LIKE` est un opérateur autorisant le remplacement d'un ou plusieurs caractères dans une chaîne de caractères (où `*` correspond à une séquence de caractères de n'importe quelle longueur (y compris vide, et `?` correspond à un seul caractère).

```
[Size] < 10000
[Compression] = "OFF"
[File system] LIKE "FAT*"
```

L'opérateur `IN` permet de spécifier une liste de valeurs:

```
IF [Cluster size] IN {512, 1024, 2048} THEN [Compression] = "Off";
IF [File system] IN {"FAT", "FAT32"} THEN [Compression] = "Off";
```

Les parties `IF`, `THEN` et `ELSE` d'une expression peuvent contenir plusieurs termes reliés par des opérateurs logiques :

`NOT` (négation)

`AND` (et)

`OR` (ou)

L'utilisation de parenthèses est autorisée afin de modifier la priorité des opérateurs par défaut.

```
IF [File system] <> "NTFS" OR
  ( [File system] = "NTFS" AND [Cluster size] > 4096 )
THEN [Compression] = "Off";

IF NOT ( [File system] = "NTFS" OR
  ( [File system] = "NTFS" AND NOT [Cluster size] <= 4096 ) )
THEN [Compression] = "Off";
```

Les paramètres peuvent être comparés avec d'autres paramètres, comme dans cet exemple :

```
#
# Machine 1
#
OS_1:   Win7, Win8, Win10
SKU_1:  Home, Pro
LANG_1: English, Spanish, Chinese

#
# Machine 2
#
OS_2:   Win7, Win8, Win10
SKU_2:  Home, Pro
LANG_2: English, Spanish, Chinese, Hindi

IF [LANG_1] = [LANG_2]
THEN [OS_1] <> [OS_2] AND [SKU_1] <> [SKU_2];
```

Contraintes Inconditionnelles (Invariants)

Un **invariant** définit une limitation toujours valide d'un domaine :

```
#
# Au moins un paramètre doit être différent pour qu'un cas de test soit
# pertinent
#

[OS_1] <> [OS_2] OR [SKU_1] <> [SKU_2] OR [LANG_1] <> [LANG_2];

#
# Tous les paramètres doivent être différents (utilisation de
# l'opérateur AND)
#

[OS_1] <> [OS_2] AND [SKU_1] <> [SKU_2] AND [LANG_1] <> [LANG_2];
```

Types

PICT utilise un système de types simple. Il n'existe que deux types de paramètres :

- Chaîne de caractères (string)
- Numérique (integer ou float)

Les types n'ont pas besoin d'être déclarés explicitement.

Un paramètre est considéré comme numérique si toutes ses valeurs peuvent être converties en nombre (entier ou flottant).

Les types sont uniquement importants lors de l'évaluation des contraintes.

Un paramètre numérique ne peut être comparé qu'à un nombre, tandis qu'un paramètre de type chaîne ne peut être comparé qu'à une autre chaîne.

Par exemple :

```
Size: 1, 2, 3, 4, 5
Value: a, b, c, d
```

```
IF [Size] > 3 THEN [Value] > "b";
```

Si une valeur possède plusieurs noms, seul le premier est pris en compte lorsque PICT détermine son type.

Sensibilité à la Casse

Par défaut, PICT effectue toutes ses comparaisons et vérifications **sans tenir compte de la casse**.

Par exemple, si deux paramètres sont définis sous les noms **OS** et **os**, une duplication sera détectée, car les noms de paramètres doivent être uniques.

De plus, les contraintes sont également résolues **sans distinction de casse** par défaut :

```
IF [OS] = "Win10" THEN ...
```

Cette condition correspondra aussi bien à `win10` qu'à `WIN10`, car les valeurs d'un paramètre ne sont pas tenues d'être uniques et, par défaut, PICT traite les comparaisons de manière **insensible à la casse**.

Cependant, l'option `/c` permet d'activer une évaluation **sensible à la casse** du modèle, ce qui signifie que `win10` et `WIN10` seraient alors considérés comme des valeurs distinctes.

Fonctionnalités Avancées de Modélisation

Réutilisation des Définitions de Paramètres

Une fois qu'un paramètre est défini, il peut être utilisé pour définir d'autres paramètres.

```
#
# Machine 1
#
OS_1:    Win7, Win8, Win10
SKU_1:   Home, Pro
LANG_1:  English, Spanish, Chinese

#
# Machine 2
#
OS_2:    <OS_1>
SKU_2:   <SKU_1>
LANG_2:  <LANG_1>, Hindi
```

Cela permet de **réduire la saisie** et d'améliorer la **maintenabilité** du modèle.

Sous-Modèles

Les sous-modèles permettent de regrouper certains paramètres en ensembles qui auront leurs propres ordres de combinaison.

Cela peut être utile lorsque certaines combinaisons de paramètres doivent être testées de manière plus approfondie, moins approfondie, ou séparément des autres paramètres du modèle.

La définition d'un sous-modèle suit le format suivant :

```
{ <NomParam1>, <NomParam2>, <NomParam3>, ... } @ <Ordre>
```

Par exemple, l'utilisation de sous-modèles est particulièrement utile lorsque des paramètres matériels et logiciels sont combinés.

Sans sous-modèles, chaque cas de test générerait une nouvelle configuration matérielle unique, ce qui pourrait être coûteux en ressources.

En regroupant tous les paramètres matériels dans un même sous-modèle, on réduit le nombre de configurations matérielles uniques, diminuant ainsi le coût des tests.

L'ordre des combinaisons attribué à chaque sous-modèle offre une flexibilité supplémentaire dans la génération des tests.

```
PLATFORM:  x86, x64, arm
CPUS:      1, 2, 4
RAM:       1GB, 4GB, 64GB
HDD:       SCSI, IDE
OS:        Win7, Win8, Win10
Browser:   Edge, Opera, Chrome, Firefox
APP:       Word, Excel, Powerpoint
```

```
{ PLATFORM, CPUS, RAM, HDD } @ 2
```

Le processus de génération des tests pour le modèle ci-dessus suit la hiérarchie suivante :

Avec l'argument /o:3, PICT générera environ 10 configurations matérielles, contre 40 configurations matérielles sans sous-modèle. Cette réduction du nombre de configurations matérielles permet d'optimiser les coûts des tests.

Cependant, cette optimisation se traduit par une augmentation du nombre total de cas de test générés :

- Environ 140 cas de test avec sous-modèle
- Environ 60 cas de test sans sous-modèle

(Tous avec /o:3.)

Remarques :

- Vous pouvez définir autant de sous-modèles que nécessaire ; un paramètre peut appartenir à plusieurs sous-modèles. Cependant, la hiérarchie du modèle ne peut être qu'à un seul niveau.
- L'ordre combinatoire d'un sous-modèle ne peut pas dépasser le nombre total de ses paramètres. Dans l'exemple ci-dessus, l'ordre du premier sous-modèle peut être entre 1 et 4.
- Si aucun ordre n'est spécifié pour un sous-modèle, c'est l'ordre défini par l'option /o qui sera appliqué.

Aliasing (Alias de Valeurs)

L'**aliasing** permet de spécifier **plusieurs noms pour une seule valeur**. Cela **ne modifie pas** la complexité combinatoire du modèle. Peu importe le nombre de noms associés à une valeur, ils sont **traités comme une seule entité**.

La seule différence réside dans la **sortie des résultats** : chaque cas de test contenant cette valeur utilisera **l'un de ses alias** à tour de rôle.

Par défaut, les noms sont **séparés par le caractère |**, mais cela peut être modifié avec l'option **/a**.

Exemple :

```
OS_1:    Win2008, Win2012, Win2016
SKU_1:    Professional, Server | Datacenter
```

Remarques :

- Lors de l'évaluation des contraintes, seul le premier nom est pris en compte.
Par exemple, `[SKU_1] = "Server"` correspondra à un match, mais `[SKU_1] = "Datacenter"` ne fonctionnera pas.
- Seul le premier nom est utilisé pour déterminer si une valeur est négative ou pour détecter son type.

Negative Testing

En plus de tester les combinaisons valides (appelées "tests positifs" ou « Positive Testing »), il est souvent souhaitable de tester des valeurs en dehors de l'intervalle autorisé afin de vérifier si le programme gère correctement les erreurs.

Les tests négatifs (« Negative Testing ») doivent être conçus de façon qu'un seul élément hors plage soit présent dans chaque cas de test.

Cela est dû au fait que la plupart des applications génèrent une erreur dès la première anomalie détectée.

Masquage des Entrées ("Input Masking")

Un problème courant dans la conception des tests est le masquage des entrées (input masking), où une valeur en dehors de l'intervalle empêche une autre valeur invalide d'être testée.

Prenons l'exemple suivant, une fonction qui prend deux arguments :

```
float SumSquareRoots( float a, float b )
{
    if ( a < 0 ) throw error;           // [1]
    if ( b < 0 ) throw error;           // [2]

    return ( sqrt( a ) + sqrt( b ) );
};
```

Bien que la routine puisse être appelée avec n'importe quel nombre pour **a** ou **b**, elle ne peut effectuer le calcul que sur des nombres **non négatifs**. C'est pourquoi des vérifications **[1]** et **[2]** sont effectuées sur les arguments.

Imaginons maintenant un test avec (**a: -1, b: -1**) pour tester le cas des nombres négatifs. Ici, **a: -1 "masque" b: -1**, car la vérification **[2]** ne sera jamais exécutée. Si cette vérification n'existait pas, le problème dans le code passerait **inaperçu**.

Éviter le Masquage des Entrées

Pour éviter ce problème de **masquage des entrées**, il est **essentiel** qu'aucune combinaison de test ne contienne **plus d'une valeur hors plage**.

Dans PICT, il est possible de marquer une valeur comme **invalid** ou **hors des limites autorisées** en la **préfixant avec ~ (tilde)**.

L'option **/n** permet également de spécifier un préfixe différent selon les besoins.

```
#
# SumSquareRoots model
#
```

```
A: ~-1, 0, 1, 2
B: ~-1, 0, 1, 2
```

PICT garantit que **toutes les paires possibles** de valeurs **valides** (ou dans les limites autorisées) seront couvertes.

De plus, **toutes les combinaisons impliquant une valeur hors plage ou invalide** seront **toujours associées uniquement à des valeurs valides**, évitant ainsi le problème du **masquage des entrées**.

A	B
2	0
2	2
0	0
0	1
1	0
2	1
0	2
1	2
1	1
~-1	2
1	~-1
0	~-1
~-1	1
2	~-1
~-1	0

Notes:

1. The ~ prefix is not a part of a value when it comes to type detection or constraint evaluation. A valid constraint for the above example would look like this: `if [A] = -1 then [B] = 0;`. The ~ prefix however will show up in the output.
2. If a value has multiple names, only prefixing the first name will make the value out-of-range.

Exclusion Complète de Paramètres – La Technique de la Valeur Factice ("Dummy Value")

Certaines suites de tests peuvent nécessiter que, sous certaines conditions, des paramètres entiers soient ignorés.

Le test négatif (décrit ci-dessus) peut être un cas d'usage : lorsqu'une erreur ou un échec arrête l'application en cours de test, les autres paramètres deviennent inutiles.

Par conception, PICT optimise le nombre de tests en minimisant le nombre de cas nécessaires pour couvrir toutes les combinaisons. Cela inclut des cas qui peuvent provoquer des échecs prématurés, entraînant une perte de combinaisons testables.

Dans certains cas, l'utilisation du préfixe ~ peut prévenir ce problème. Toutefois, lorsque cela ne suffit pas, on peut utiliser la technique de la valeur factice ("dummy value technique").

Implémentation de la Technique de la Valeur Factice

PICT ne gère pas automatiquement cette technique, vous devez donc modifier manuellement votre modèle.

Prenons l'exemple suivant, où P1 = -1 déclenche une erreur dans l'application :

```
P1: -1, 0, 1
P2: A, B, C
P3: X, Y, Z
```

Nous souhaitons tester cette condition de manière **indépendante**, sans associer de valeurs aux paramètres **P2** ou **P3** lorsqu'une erreur est déclenchée par **P1 = -1**.

Solution : Utilisation d'une Valeur Factice (NA)

Pour cela, nous ajoutons une **valeur factice** (NA) aux paramètres **P2** et **P3**.

Cette valeur NA servira d'indicateur pour signifier que le paramètre doit être **ignoré**.

Ensuite, nous définissons des **contraintes** afin de garantir que les tests sont générés correctement :

```
P1: -1, 0, 1
P2: A, B, C, NA
P3: X, Y, Z, NA
```

```
IF [P1] = -1
  THEN [P2] = "NA" AND [P3] = "NA"
  ELSE [P2] <> "NA" AND [P3] <> "NA";
```

Cela aboutira à un seul cas de test pour $P1 = -1$, où $P2$ et $P3$ seront définis à NA.

Aucune autre valeur de $P2$ et $P3$ ne sera utilisée dans le test contenant $P1 = -1$, évitant ainsi de gaspiller des combinaisons inutiles.

P1	P2	P3
0	C	Z
1	B	Z
1	C	Y
0	A	X
1	C	X
0	B	X
0	A	Y
-1	NA	NA
0	B	Y
1	A	Z

L'inconvénient de cette méthode est que les **contraintes deviennent progressivement plus complexes** à mesure que des **valeurs supplémentaires déclenchant des échecs** sont ajoutées au modèle.

Par exemple, si l'on ajoute une nouvelle valeur **P2: Null**, il faudra ajuster les contraintes pour s'assurer que les tests sont générés correctement.

```
P1: -1, 0, 1
P2: A, B, C, Null, NA
P3: X, Y, Z, NA
```

```
IF [P1] = -1
  THEN [P2] = "NA" AND [P3] = "NA"
  ELSE [P2] <> "NA" AND
    ( [P3] <> "NA" OR [P2] = "Null" );
```

```
IF [P2] = "Null" OR [P2] = "NA"
  THEN [P3] = "NA";
```

Deux tests supplémentaires seront ajoutés (toute la sortie n'est pas affichée) :

P1	P2	P3
0	Null	NA
1	Null	NA

Voici un exemple plus concret :

```
OS: Windows, Ubuntu
CPU: Intel, AMD
DBMS: PostgreSQL, MySQL
JavaVersion: 18, 19, 20
DotNetVersion: 4.8, 4.8.1
```

Le modèle ci-dessus générera les tests suivants :

OS	CPU	DBMS	JavaVersion	DotNetVersion
Windows	AMD	PostgreSQL	18	4.8
Windows	Intel	MySQL	20	4.8.1
Ubuntu	Intel	PostgreSQL	19	4.8.1
Ubuntu	AMD	MySQL	20	4.8
Windows	AMD	MySQL	19	4.8
Ubuntu	AMD	MySQL	18	4.8.1
Windows	Intel	MySQL	18	4.8
Ubuntu	Intel	PostgreSQL	20	4.8

Le problème est que .NET n'existe pas sur Ubuntu, donc le paramètre **DotNetVersion** n'a aucun sens pour ces tests.

Si nous ajoutons "NA" et une contrainte au modèle, comme ceci :

```
OS: Windows, Ubuntu
CPU: Intel, AMD
DBMS: PostgreSQL, MySQL
JavaVersion: 18, 19, 20
DotNetVersion: 4.8, 4.8.1, NA
```

```
IF [OS] = "Ubuntu"
  THEN [DotNetVersion] = "NA"
  ELSE [DotNetVersion] <> "NA";
```

...alors les tests suivants seront générés :

OS	CPU	DBMS	JavaVersion	DotNetVersion
Windows	AMD	PostgreSQL	20	4.8
Ubuntu	Intel	MySQL	18	NA
Windows	Intel	PostgreSQL	18	4.8
Windows	AMD	MySQL	18	4.8.1
Windows	AMD	MySQL	19	4.8
Ubuntu	AMD	PostgreSQL	19	NA
Windows	Intel	PostgreSQL	19	4.8.1
Windows	Intel	MySQL	20	4.8.1
Ubuntu	AMD	PostgreSQL	20	NA

Pondération (« Weighting »)

Les pondérations indiquent au générateur de privilégier certaines valeurs de paramètres par rapport à d'autres. Les pondérations sont des **entiers positifs**. Lorsqu'elles ne sont pas spécifiées, les valeurs ont une pondération par défaut de **1**.

```
#
# Concentrons-nous sur les partitions primaires formatées en NTFS #
```

```
Type:           Primary (10), Logical, Single, Span, Stripe, Mirror,
RAID-5
Format method:  quick, slow
File system:    FAT, FAT32, NTFS (10)
```

Les pondérations ne sont pas garanties, mais servent uniquement d'indicateurs opportunistes. Il n'est pas certain que les pondérations spécifiées seront respectées.

L'objectif principal de PICT est de couvrir toutes les combinaisons de valeurs avec un nombre minimal de cas de test. Les pondérations ne seront prises en compte que lorsque les choix de valeurs n'affectent pas ce critère de couverture.

Pour cette raison, les valeurs de pondération n'ont pas de signification intuitive.

Par exemple, quand un paramètre est défini ainsi :

```
File system:      FAT, FAT32, NTFS (10)
```

Cela ne signifie pas que NTFS apparaîtra dix fois plus souvent que FAT ou FAT32 dans la sortie générée. Cela signifie simplement que NTFS a **dix fois plus de chances d'être sélectionné lorsqu'il n'affecte pas la couverture globale**.

Seeding (Ensemencement)

Le seeding permet deux scénarios :

1. Spécifier des combinaisons importantes, par exemple celles pouvant entraîner des régressions, qui doivent impérativement être incluses dans toute suite de tests générée. Dans ce cas, PICT initialisera la sortie avec les combinaisons fournies et complètera la suite de tests en s'appuyant sur ces données.
2. Minimiser les modifications de la sortie lorsque le modèle est mis à jour. En fournissant à PICT les résultats d'une génération de tests précédente, l'outil tentera de réutiliser autant que possible les anciens cas de test.

Les lignes de seeding (ensemencement) doivent être définies dans un fichier séparé. Utilisez l'option /e pour spécifier l'emplacement du fichier de seeding :

```
pict.exe model.txt /e:seedrows.txt
```

Les fichiers de seeding ont le même format que toute sortie PICT.

- La première ligne contient les noms des paramètres, séparés par des tabulations.
- Chaque ligne suivante représente un cas de test avec des valeurs également séparées par tabulations.

Ce format peut être préparé manuellement ou bien issu d'une sortie précédente de PICT et réutilisé directement.

Ver	SKU	Lang	Arch
Win7	Pro	EN	x86
Win7		FR	x86
Win10	Pro	EN	x64

Une ligne de seeding (ensemencement) peut être complète, c'est-à-dire avec des valeurs spécifiées pour tous les paramètres.

Elle peut aussi être partielle.

Par exemple, dans l'exemple ci-dessus, la deuxième ligne de seeding ne contient pas de valeur pour le paramètre `SKU`. Dans ce cas, le processus de génération sélectionnera automatiquement la meilleure valeur pour `SKU`.

Règles d'association des lignes de seeding avec le modèle actuel :

1. Si un fichier de seeding contient un paramètre qui n'existe pas dans le modèle actuel, la colonne correspondante dans le fichier de seeding sera ignorée. Les paramètres sont associés par leur nom.
2. Si une ligne de seeding contient une valeur qui n'existe pas dans le modèle actuel, cette valeur sera ignorée, mais le reste de la ligne sera toujours utilisé si elle reste valide. Une ligne complète peut ainsi devenir partielle.
3. Si une ligne de seeding viole une contrainte du modèle, elle sera totalement ignorée.
4. PICT émettra des avertissements si les cas (1) ou (2) se produisent.

De plus, certaines pratiques, bien qu'autorisées dans les modèles PICT, peuvent créer des ambiguïtés lorsqu'on utilise le seeding :

- Noms de paramètres et de valeurs vides.
- Noms de paramètres et de valeurs contenant des caractères de tabulation.
- Valeurs et alias non uniques au sein d'un même paramètre.

Il est préférable d'éviter ces situations.

PICT émettra un avertissement si de tels problèmes sont détectés dans votre modèle lors de l'utilisation du seeding.

Grammaire spécifique aux contraintes

Ces règles définissent comment les contraintes sont formulées et évaluées dans PICT.

```
Constraints    :: =  
  Constraint  
| Constraint Constraints
```

```
Constraint     :: =  
  IF Predicate THEN Predicate ELSE Predicate;  
| Predicate;
```

```
Predicate      :: =  
  Clause  
| Clause LogicalOperator Predicate
```

```
Clause         :: =  
  Term  
| ( Predicate )  
| NOT Predicate
```

```
Term          :: =
  ParameterName Relation Value
| ParameterName LIKE PatternString
| ParameterName IN { ValueSet }
| ParameterName Relation ParameterName
```

```
ValueSet      :: =
  Value
| Value, ValueSet
```

```
LogicalOperator ::=
  AND
| OR
```

```
Relation      :: =
  =
| <>
| >
| >=
| <
| <=
```

```
ParameterName ::= [String]
```

```
Value         :: =
  "String"
| Number
```

```
String        :: = chaîne de caractères
```

```
Number        :: = nombre
```

```
PatternString ::= s chaîne avec des caractères génériques (* pour
n'importe quelle séquence, ? pour un seul caractère)
```